

SHETH LUJ COLLEGE

AI PRACTICAL – 2

A* Search and Recursive Best-First Search

- Implement the A* Search algorithm for solving a pathfinding problem.
- Implement the Recursive Best-First Search algorithm for the same problem.
- Compare the performance and effectiveness of both algorithms.

Code:-

```
import heapq
```

```
import matplotlib.pyplot as plt
```

```
import networkx as nx
```

```
# -----
```

```
# 1. DATA STRUCTURE POPULATION
```

```
# -----
```

```
graph = {
```

```
    'MVLU College': {'Bandra': 12, 'Sion': 14},
```

```
    'Bandra': {'Worli': 6, 'Byculla': 10},
```

```
    'Sion': {'Byculla': 13},
```

```
    'Worli': {'Byculla': 5, 'CSMT': 9},
```

```
    'Byculla': {'CSMT': 6},
```

```
    'CSMT': {'Marine Lines': 2},
```

```
    'Marine Lines': {}
```

```
}
```

```
heuristics = {
```

```
    'MVLU College': 20,
```

```
    'Bandra': 12,
```

```
    'Sion': 11,
```

```
    'Worli': 7,
```

```
    'Byculla': 5,
```

```
    'CSMT': 2,
```

DANIYAL KHAN

T088

SHETH LUJ COLLEGE

AI PRACTICAL – 2

```
'Marine Lines': 0  
}
```

```
node_positions = {  
    'MVLU College': (2, 8),  
    'Bandra': (1, 6),  
    'Sion': (3, 6),  
    'Worli': (1, 4),  
    'Byculla': (3, 4),  
    'CSMT': (2, 2),  
    'Marine Lines': (2, 0)  
}
```

```
# -----  
# 2. A* SEARCH ALGORITHM  
# -----
```

```
def a_star_search(graph, heuristics, start, goal):  
    priority_queue = [(heuristics[start], start, [start], 0)]  
    visited = set()  
  
    while priority_queue:  
        f_score, current, path, g_score = heapq.heappop(priority_queue)  
  
        if current in visited:  
            continue  
  
        visited.add(current)  
  
        if current == goal:
```

SHETH LUJ COLLEGE

AI PRACTICAL – 2

```
return path, g_score
```

```
for neighbor, edge_weight in graph[current].items():
```

```
    if neighbor not in visited:
```

```
        next_g = g_score + edge_weight
```

```
        next_f = next_g + heuristics[neighbor]
```

```
        heapq.heappush(
```

```
            priority_queue,
```

```
            (next_f, neighbor, path + [neighbor], next_g)
```

```
        )
```

```
return None, float('inf')
```

```
# -----
```

```
# RUN THE ALGORITHM
```

```
# -----
```

```
start = "MVLU College"
```

```
goal = "Marine Lines"
```

```
optimal_path, total_distance = a_star_search(
```

```
    graph,
```

```
    heuristics,
```

```
    start,
```

```
    goal
```

```
)
```

```
# -----
```

```
# DISPLAY OUTPUT
```

```
# -----
```

DANIYAL KHAN

T088

SHETH LUJ COLLEGE

AI PRACTICAL – 2

```
print("=" * 70)

print("          A* SEARCH ALGORITHM")

print("=" * 70)


print(f"Start Node : {start}")
print(f"Goal Node : {goal}")
print()


print("Optimal Path:")
print(" -> ".join(optimal_path))
print()


print(f"Total Distance : {total_distance} km")
print()


print("=" * 70)
print("{:<5}{:<20}{:<15}".format("No.", "Location", "Heuristic"))
print("=" * 70)


for i, node in enumerate(optimal_path, start=1):
    print("{:<5}{:<20}{:<15}".format(i, node, heuristics[node]))

print("=" * 70)


# -----

# 3. GRAPH VISUALIZATION

# -----


G = nx.DiGraph()
```

DANIYAL KHAN
T088

SHETH LUJ COLLEGE

AI PRACTICAL – 2

```
for node, neighbors in graph.items():
```

```
    for neighbor, weight in neighbors.items():
```

```
        G.add_edge(node, neighbor, weight=weight)
```

```
plt.figure(figsize=(10, 8))
```

```
path_edges = list(zip(optimal_path[:-1], optimal_path[1:]))
```

```
normal_edges = [edge for edge in G.edges() if edge not in path_edges]
```

```
nx.draw_networkx_nodes(
```

```
    G,
```

```
    node_positions,
```

```
    node_size=2500,
```

```
    node_color='lightblue'
```

```
)
```

```
nx.draw_networkx_edges(
```

```
    G,
```

```
    node_positions,
```

```
    edgelist=normal_edges,
```

```
    edge_color='gray',
```

```
    width=1.5,
```

```
    arrows=True,
```

```
    arrowsize=20
```

```
)
```

```
nx.draw_networkx_edges(
```

```
    G,
```

```
    node_positions,
```

SHETH LUJ COLLEGE

AI PRACTICAL – 2

```
edgelist=path_edges,
edge_color='darkorange',
width=3.5,
arrows=True,
arrowsize=20
)

node_labels = {
    node: f"{node}\nh(n)={heuristics[node]}"
    for node in G.nodes()
}

nx.draw_networkx_labels(
    G,
    node_positions,
    labels=node_labels,
    font_size=9,
    font_weight='bold'
)

edge_labels = nx.get_edge_attributes(G, 'weight')

formatted_edge_labels = {
    edge: f"{weight} km"
    for edge, weight in edge_labels.items()
}

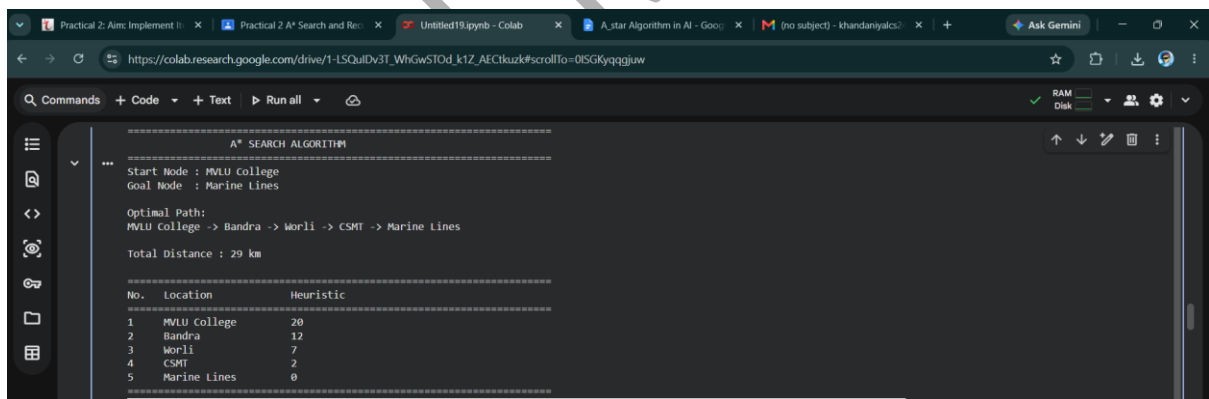
nx.draw_networkx_edge_labels(
    G,
    node_positions,
```

SHETH LUJ COLLEGE

AI PRACTICAL – 2

```
edge_labels=formatted_edge_labels,  
font_color='red',  
font_size=8  
)  
  
plt.title(  
    "A* Routing Map: MVLU College to Marine Lines\    "Highlighted Orange Path = Optimal Route",  
    fontsize=14  
)  
  
plt.axis("off")  
plt.tight_layout()  
plt.show()
```

Output:-



```
=====
A* SEARCH ALGORITHM
=====
Start Node : MVLU College
Goal Node : Marine Lines

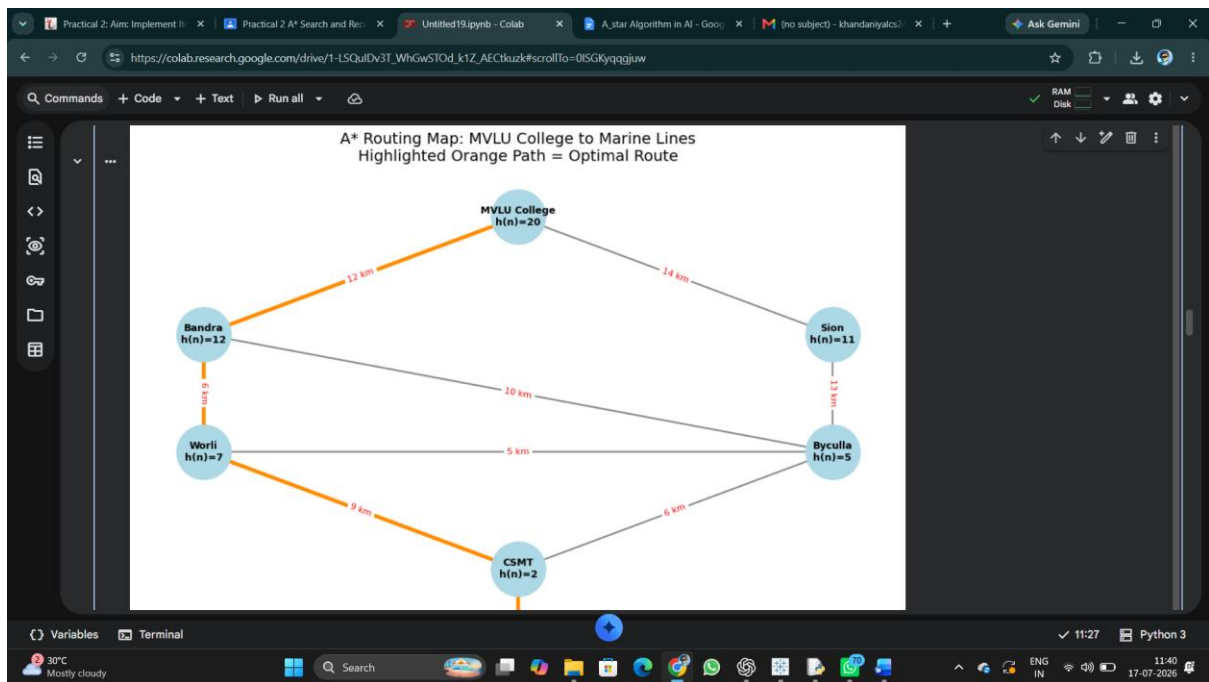
Optimal Path:
MVLU College -> Bandra -> Worli -> CSMT -> Marine Lines

Total Distance : 29 km

=====
No.  Location      Heuristic
=====
1    MVLU College   20
2    Bandra         12
3    Worli          7
4    CSMT           2
5    Marine Lines   0
=====
```

SHETH LUJ COLLEGE

AI PRACTICAL – 2



Code:-

```
import matplotlib.pyplot as plt
```

```
# -----
```

```
# 1. NETWORK DEFINITION
```

```
# -----
```

```
# Road distances (g)
```

```
graph = {
    "MVLU College": {"Andheri": 4, "Jogeshwari": 3},
    "Andheri": {"Borivali": 15, "BKC": 10},
    "Jogeshwari": {"Borivali": 14},
    "Borivali": {"Thane": 20},
    "BKC": {"Thane": 18},
    "Thane": {"Panvel": 35},
    "Panvel": {"Khopoli": 40},
    "Khopoli": {"Lonavala": 18},
```

SHETH LUJ COLLEGE

AI PRACTICAL – 2

```
"Lonavala": {"Bhushi Dam": 4},  
"Bhushi Dam": {}  
}
```

Heuristic values (h)

```
heuristics = {  
    "MVLU College": 130,  
    "Andheri": 120,  
    "Jogeshwari": 118,  
    "Borivali": 100,  
    "BKC": 95,  
    "Thane": 75,  
    "Panvel": 45,  
    "Khopoli": 18,  
    "Lonavala": 4,  
    "Bhushi Dam": 0  
}
```

Coordinates for drawing

```
coords = {  
    "MVLU College": (0, 10),  
    "Andheri": (-2, 8),  
    "Jogeshwari": (2, 8),  
    "Borivali": (0, 6),  
    "BKC": (-2, 5),  
    "Thane": (0, 4),  
    "Panvel": (2, 2),  
    "Khopoli": (4, 0),  
    "Lonavala": (6, -2),  
    "Bhushi Dam": (8, -4)
```

SHETH LUJ COLLEGE

AI PRACTICAL – 2

```
}
```

```
# -----
```

```
# 2. RBFS ALGORITHM
```

```
# -----
```

```
def rbfs_search(start, goal):
```

```
    success, path, cost, _ = rbfs(
```

```
        start,
```

```
        goal,
```

```
        g=0,
```

```
        f_limit=float("inf"),
```

```
        path=[start]
```

```
    )
```

```
    return path, cost
```

```
def rbfs(node, goal, g, f_limit, path):
```

```
    if node == goal:
```

```
        return True, path, g, g
```

```
    neighbours = graph[node]
```

```
    if not neighbours:
```

```
        return False, [], 0, float("inf")
```

```
    successors = []
```

```
    for neighbour, distance in neighbours.items():
```

SHETH LUJ COLLEGE

AI PRACTICAL – 2

if neighbour not in path:

```
new_g = g + distance
```

```
new_f = max(new_g + heuristics[neighbour],  
            g + heuristics[node])
```

```
successors.append([new_f, neighbour, new_g])
```

if not successors:

```
return False, [], 0, float("inf")
```

while True:

```
successors.sort(key=lambda x: x[0])
```

```
best = successors[0]
```

```
if best[0] > f_limit:
```

```
    return False, [], 0, best[0]
```

```
alternative = successors[1][0] if len(successors) > 1 else float("inf")
```

```
success, result_path, total_cost, returned_f = rbfs(  
    best[1],  
    goal,  
    best[2],  
    min(f_limit, alternative),  
    path + [best[1]]  
)
```

SHETH LUJ COLLEGE

AI PRACTICAL – 2

```
best[0] = returned_f
```

```
if success:
```

```
    return True, result_path, total_cost, returned_f
```

```
# -----
```

```
# 3. RUN RBFS
```

```
# -----
```

```
start = "MVLU College"
```

```
goal = "Bhushi Dam"
```

```
path, total_dist = rbfs_search(start, goal)
```

```
print("=" * 70)
```

```
print("    RECURSIVE BEST-FIRST SEARCH (RBFS)")
```

```
print("=" * 70)
```

```
print("Start :", start)
```

```
print("Goal :", goal)
```

```
print()
```

```
print("Optimal Path:")
```

```
print(" -> ".join(path))
```

```
print()
```

```
print("Total Distance:", total_dist, "km")
```

```
print("\nPath Details")
```

```
print("-" * 70)
```

```
print("{:<5}{:<20}{:<12}".format("No.", "Location", "Heuristic"))
```

DANIYAL KHAN
T088

SHETH LUJ COLLEGE

AI PRACTICAL – 2

```
print("-" * 70)
```

```
for i, node in enumerate(path, start=1):
```

```
    print("{:<5} {:<20} {:<12}".format(i, node, heuristics[node]))
```

```
print("-" * 70)
```

```
# -----
```

```
# 4. DRAW GRAPH
```

```
# -----
```

```
plt.figure(figsize=(12, 8))
```

```
for node, neighbours in graph.items():
```

```
    x1, y1 = coords[node]
```

```
    for neighbour, dist in neighbours.items():
```

```
        x2, y2 = coords[neighbour]
```

```
        is_path = (
```

```
            node in path and
```

```
            neighbour in path and
```

```
            path.index(neighbour) == path.index(node) + 1
```

```
        )
```

```
        color = "red" if is_path else "gray"
```

```
        width = 4 if is_path else 2
```

SHETH LUJ COLLEGE

AI PRACTICAL – 2

```
plt.plot(  
    [x1, x2],  
    [y1, y2],  
    color=color,  
    linewidth=width  
)
```

```
plt.text(  
    (x1 + x2) / 2,  
    (y1 + y2) / 2,  
    str(dist) + " km",  
    color="blue",  
    fontsize=9  
)
```

```
for node, (x, y) in coords.items():
```

```
    color = "orange" if node in path else "skyblue"
```

```
plt.scatter(  
    x,  
    y,  
    s=1800,  
    color=color,  
    edgecolors="black"  
)
```

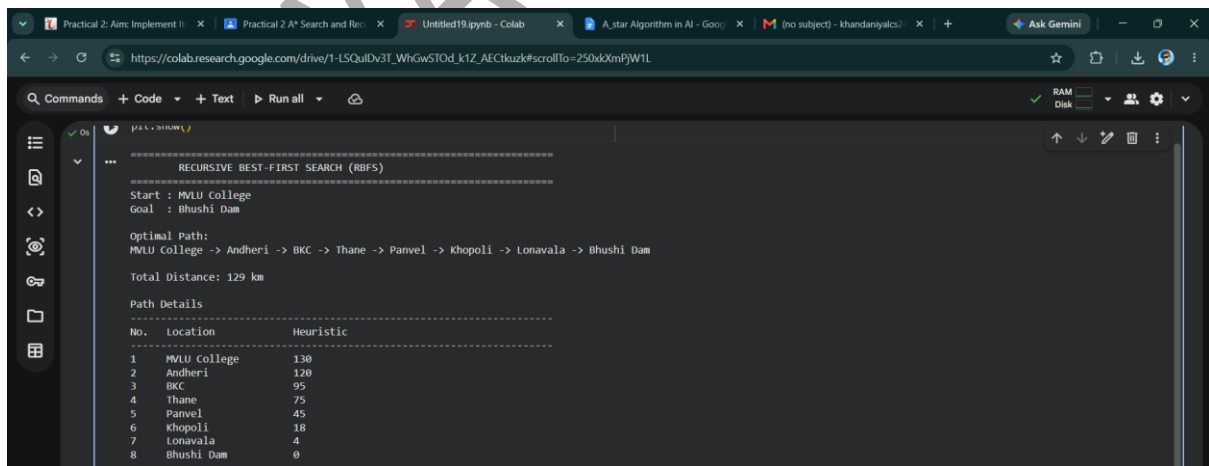
```
plt.text(  
    x,  
    y,
```

SHETH LUJ COLLEGE

AI PRACTICAL – 2

```
f"{node}\nh={heuristics[node]}",  
  
ha="center",  
  
va="center",  
  
fontsize=8,  
  
fontweight="bold"  
  
)  
  
plt.title(  
  
    "Recursive Best-First Search (RBFS)\nMVLU College → Bhushi Dam (Lonavala)",  
  
    fontsize=14,  
  
    fontweight="bold"  
  
)  
  
plt.axis("off")  
plt.tight_layout()  
plt.show()
```

output:-



```
RECURSIVE BEST-FIRST SEARCH (RBFS)  
-----  
Start : MVLU College  
Goal  : Bhushi Dam  
  
Optimal Path:  
MVLU College -> Andheri -> BKC -> Thane -> Panvel -> Khopoli -> Lonavala -> Bhushi Dam  
  
Total Distance: 129 km  
  
Path Details  
-----  
No.  Location      Heuristic  
-----  
1    MVLU College   130  
2    Andheri        120  
3    BKC            95  
4    Thane          75  
5    Panvel         45  
6    Khopoli        18  
7    Lonavala       4  
8    Bhushi Dam     0  
-----
```

SHETH LUJ COLLEGE

AI PRACTICAL – 2

